

# Python Functions – Ultimate Coding Exam Sheet

---

Topics covered: Functions, Arguments, \*args, \*\*kwargs, Scope, Decorators, Lambda, Recursion, and Generators.

Difficulty progression: Easy → Medium → Hard → Mixed Challenges

Rule: Attempt each question yourself before checking any solution. Focus on logic, edge cases, and correct use of return/yield.

## SECTION 1: BASIC FUNCTIONS EASY

**Q1.** Create a function to print: Hello, Soumya!

Your answer/code:

**Q2.** Create a function `greet()` that takes a name and prints: `Hello <name>`. Example:  
`greet("Soumya") → Hello Soumya`

Your answer/code:

**Q3.** Create a function `add(a, b)` that returns the sum.

Your answer/code:

**Q4.** Create a function that takes a number and returns whether it is even or odd. Example:  
`check_even_odd(7)` → Odd.

Your answer/code:

**Q5.** Create a function `square(n)` that returns the square.

Your answer/code:

**Q6.** Create a function that takes a list and returns the largest element. Do not use `max()`.

Your answer/code:

**Q7.** Create a function that counts the number of vowels in a string. Example: "Soumya" → 3.

Your answer/code:

**Q8.** Create a function that calculates factorial. Example: factorial(5) → 120.

Your answer/code:

**Q9.** Create a function that checks whether a number is prime.

Your answer/code:

**Q10.** Create a function that takes a string and returns the reversed string. Do not use slicing `[::-1]`.

Your answer/code:

## SECTION 2: RETURN VS PRINT

**Q11.** Predict the output and explain why:

```
def add(a, b):  
    print(a + b)
```

```
x = add(2, 3)  
print(x)
```

Your answer/code:

**Q12.** Fix this code:

```
def multiply(a, b):  
    print(a * b)
```

```
result = multiply(4, 5)  
print(result + 10)
```

Your answer/code:

**Q13.** Create a function that returns the sum, difference, and product of two numbers.

Example: `result = calculator(10, 5);`  
`print(result).`

Your answer/code:

## SECTION 3: FUNCTION ARGUMENTS

**Q14.** Create `def student(name, age, city):` and call it using positional arguments.

Your answer/code:

**Q15.** Create the same student function and call it using keyword arguments:  
`student(name="Soumya", age=22, city="Jodhpur")`.

Your answer/code:

**Q16.** Create `greet(name="Guest")`. Calling `greet()` should print Hello Guest, and `greet("Soumya")` should print Hello Soumya.

Your answer/code:

**Q17.** Create `calculate_bill(price, quantity=1)` and return `price × quantity`.

Your answer/code:

**Q18.** Predict the output:

```
def show(name, age=20):  
    print(name, age)
```

```
show("Soumya")  
show("Soumya", 25)
```

Your answer/code:

**Q19.** Explain why `def test(a=10, b):` is invalid. Correct it.

Your answer/code:

## **SECTION 4: \*ARGS** 🟡

**Q20.** Create `add_numbers(*numbers)` that returns the sum of any number of arguments.

Your answer/code:

**Q21.** Create a function that finds the largest number using `*args`. Do not use `max()`.

Your answer/code:

**Q22.** Create `average(*numbers)` that returns the average.

Your answer/code:

**Q23.** Predict what is stored inside `numbers` and identify its datatype:

```
def test(*numbers):  
    print(numbers)
```

```
test(10, 20, 30)
```

Your answer/code:

**Q24.** Create def student(name, \*marks): that prints Name, Marks, and Average.

Your answer/code:

## SECTION 5: \*\*KWARGS 🟡

**Q25.** Create `student_info(**details)`. Pass `name='Soumya'`, `age=22`, `city='Jodhpur'` and print all key-value pairs.

Your answer/code:

**Q26.** Create a function that receives unlimited product details. Example:

`product_info(name='Laptop', price=50000, brand='HP')`.

Your answer/code:

**Q27.** What is the datatype of kwargs?

```
def test(**kwargs):  
    print(kwargs)
```

Your answer/code:

**Q28.** Create `def student(name, age=20, *subjects, **details):`. Call it with all possible argument types and explain what gets stored where.

Your answer/code:

## SECTION 6: ARGUMENT ORDER 🟡🔥

**Q29.** Put these in correct order: normal arguments, default arguments, \*args, \*\*kwargs. Then write a valid function.

Your answer/code:

**Q30.** Create `def demo(a, b=10, *args, **kwargs)`: and call it in at least 3 different ways. Predict each output before running.

Your answer/code:

## SECTION 7: SCOPE 🟡

**Q31.** Will this work? Explain why:

```
def test():  
    x = 10
```

```
test()  
print(x)
```

Your answer/code:

**Q32.** Explain this global variable example:

```
x = 10
```

```
def test():  
    print(x)
```

```
test()
```

Your answer/code:

**Q33.** Predict the output:

```
x = 10
```

```
def test():  
    x = 20  
    print(x)
```

```
test()  
print(x)
```

Your answer/code:

**Q34.** Create a global variable count = 0.

Create a function that increases it by 1 every time the function is called.

Your answer/code:

**Q35.** Create a function inside another function and access the outer function's variable.

Your answer/code:

**Q36.** Create `outer()` with `count = 0` and `inner()` that increases count using `nonlocal`.

Your answer/code:

## SECTION 9: RECURSION 🟡🔥

**Q37.** Write factorial using recursion.

Your answer/code:

**Q38.** Write the sum of numbers from 1 to n using recursion. Example:  $\text{sum}(5) = 15$ .

Your answer/code:

**Q39.** Write Fibonacci using recursion.

Your answer/code:

**Q46.** Find the sum of elements in a list using recursion. Example:  $[1, 2, 3, 4, 5] \rightarrow 15$ .

Your answer/code:

**Q47.** Reverse a string using recursion.

Example: python  $\rightarrow$  nohtyp.

Your answer/code:

**Q48.** Count the digits of a number using recursion. Example: 12345  $\rightarrow$  5.

Your answer/code:

**Q49.** Find whether a string is a palindrome using recursion. Examples: madam  $\rightarrow$  True, python  $\rightarrow$  False.

Your answer/code:

**Q50.** Find the maximum number in a list using recursion. Do not use max().

Your answer/code:

## SECTION 10: GENERATORS 🌞🔥

**Q51.** Create a generator that yields numbers from 1 to n using yield.

Your answer/code:

**Q52.** Create a generator that yields only even numbers.

Your answer/code:

**Q53.** Create a Fibonacci generator that yields:  
0, 1, 1, 2, 3, 5, 8, ...

Your answer/code:

**Q54.** Explain the difference between return and yield with code.

Your answer/code:

**Q55.** Predict the output:

```
def test():
```

```
    yield 1
```

```
    yield 2
```

```
    yield 3
```

```
x = test()
```

```
print(next(x))
```

```
print(next(x))
```

Your answer/code:

**Q56.** What happens when `next(x)` is called one extra time after the generator is finished?

Your answer/code:

**Q57.** Create an infinite generator that yields 1, 2, 3, 4, ... How will you safely print only the first 10 values?

Your answer/code:

## SECTION 11: DECORATORS 🔥

**Q58.** Create a simple decorator that prints 'Before function', calls the original function, and then prints 'After function'.

Your answer/code:

**Q59.** Apply the decorator to:

```
def greet():  
    print('Hello')
```

Your answer/code:

**Q60.** Create a decorator that works with functions having arguments. Example:  
`@decorator def greet(name): print(name)`. Use `*args` and `**kwargs`.

Your answer/code:

**Q61.** Create a decorator that prints the function name before running it. Example:  
Running function: add.

Your answer/code:

**Q62.** Create a decorator that measures how many times a function is called. Example:  
Function called: 1 times, 2 times, 3 times.

Your answer/code:

**Q63.** Create a decorator that checks if the user is logged in. If `logged_in = True`, allow access; otherwise print Access Denied.

Your answer/code:

## SECTION 12: MIXED FUNCTION CHALLENGES



**Q64.** Create one function that accepts: required name, default city='Jodhpur', unlimited positional \*marks, and unlimited keyword \*\*details. Calculate average marks and print all information.

Your answer/code:

**Q65.** Create `analyze_numbers(*numbers)` that returns a dictionary containing `sum`, `average`, `largest`, `smallest`, `even_count`, and `odd_count`. Do not use `sum()`, `max()`, or `min()`.

Your answer/code:

**Q66.** Create a function that takes a list of numbers and uses `lambda`, `filter()`, and `map()` to return squares of only even numbers.  
Example: `[1,2,3,4,5,6] → [4,16,36]`.

Your answer/code:

**Q67.** Create a generator that receives a list and yields only prime numbers.

Your answer/code:

**Q68.** Create a recursive `flatten_list()`. Input: `[1, [2, 3], [4, [5, 6]]]` Output: `[1, 2, 3, 4, 5, 6]`.

Your answer/code:

**Q69.** Create `@validate_positive`. It should allow a function to run only if all numeric arguments are positive. Example:

`@validate_positive` def divide(a, b): return a / b. If divide(-5, 2), print Only positive numbers allowed.

Your answer/code:

**FINAL FUNCTIONS MASTER CHALLENGE** 🏆🔥



**Q70.** Create a MINI STUDENT ANALYSIS SYSTEM using as many learned concepts as

possible.

## Requirements:

1. Create `analyze_student(name, *marks, **details)`.
2. Use `*args` to accept unlimited marks.
3. Use `**kwargs` to accept city, age, and course.
4. Calculate total, average, highest mark, lowest mark, and pass/fail.
5. Use `lambda` to determine pass/fail.
6. Use a generator that yields each mark one by one.
7. Use a decorator that prints `'----- STUDENT ANALYSIS STARTED -----'` before the function and `'----- ANALYSIS COMPLETED -----'` after it.
8. Maintain a global variable

`total_students_analyzed = 0` and increase it every time the analysis function runs.

Your answer/code:

## FUNCTION MASTERY TEST

If you can solve Q1–Q70 yourself, especially Q28, Q34–36, Q41, Q49–50, Q57, Q60–63, and Q64–70, your Python functions foundation will be genuinely strong.

Suggested plan: Day 1 → Q1–20 | Day 2 → Q21–42 | Day 3 → Q43–57 | Day 4 → Q58–70